CPU EDUCATION

COMP5310

Week10 Decision Tree

TUTOR：Arya 学姐　　时长：2 小时

本周内容预览：

1.决策树

2.纯度

3.ID3 算法

4. 评估设置

5. 集成树

6. 课后习题

1. 决策树

决策树（Decision Tree）是一种常用的机器学习算法，广泛应用于分类和回归问题。

决策树通过树状结构来表示决策过程，每个内部节点代表一个特征或属性的测试，每个分支代表测试的结果，每个叶节点代表一个类别或值。

1.1 参数模型与非参数模型

参数模型 (Parametric Models) 和 非参数模型 (Non-Parametric Models) 的核心区别在于：模型在训练前是否预先假设了固定的函数形式和参数数量。

1.1.1 参数模型 (Parametric Models)

参数模型在训练开始前就预先定义了模型的结构（函数形式）和参数的数量。无论你的数据集有多大，模型的参数数量大致保持不变。

线性回归 (Linear Regression):

- 假设：数据符合直线或超平面关系。
- 参数：权重 θ 和截距 α 。无论数据有 100 行还是 100 万行，参数数量只取决于特征数量。

逻辑回归 (Logistic Regression) :

- 假设：通过 Sigmoid 函数将线性组合映射为概率。
- 参数：同样是一组固定的权重 θ 。

多项式回归 (Polynomial Regression) :

- 注意：虽然曲线是弯曲的，但它对参数 θ 是线性的（即 θ 只出现在一次方项中），因此它依然属于广义上的参数模型/线性模型。

1.1.2 非参数模型 (Non-Parametric Models)

非参数模型不对数据的函数形式做预先假设，也不预先固定参数的数量。模型的复杂度会随着训练数据的增加而增加。

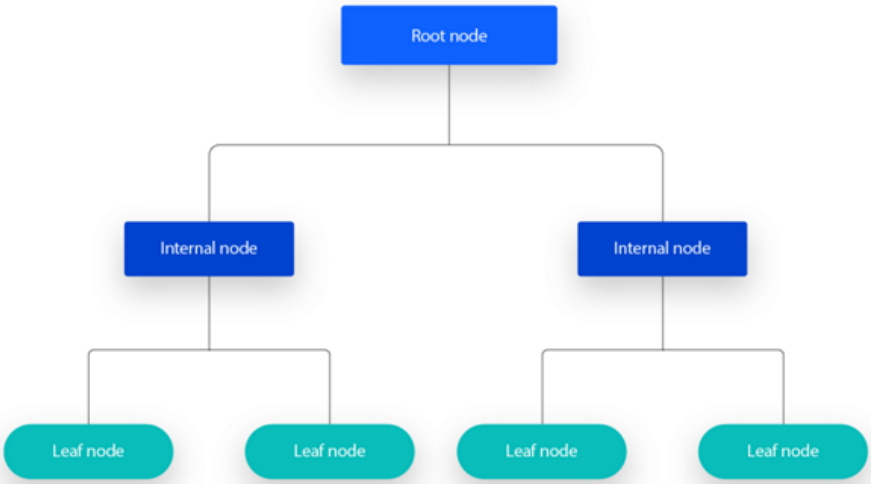
决策树 (Decision Tree):

- 机制：通过递归地根据特征划分数据。
- 参数特性：树的深度、节点数量完全取决于数据的复杂程度。数据越多、越复杂，生成的树就越深、节点越多（参数越多）。

盗版必究索赔

1.2 基本概念

节点（Node）：树中的每个点称为节点。根节点是树的起点，内部节点是决策点，叶节点是最终的决策结果。
分支（Branch）：从一个节点到另一个节点的路径称为分支。
分裂（Split）：根据某个特征将数据集分成多个子集的过程。
纯度（Purity）：衡量一个子集中样本的类别是否一致。纯度越高，说明子集中的样本越相似。



1.3 决策树的优缺点

优点

- 易于理解和解释：决策树的结构直观，易于理解和解释。
- 处理多种数据类型：可以处理数值型和类别型数据。
- 不需要数据标准化：决策树不需要对数据进行标准化或归一化处理。

缺点

- 容易过拟合：决策树容易过拟合，特别是在数据集较小或树深度较大时。
- 对噪声敏感：决策树对噪声数据较为敏感，可能导致模型性能下降。
- 不稳定：数据的小变化可能导致生成完全不同的树。

1.4 工作原理

决策树的构建过程通常被称为 “分而治之” (Divide and Conquer)。最著名的算法有 ID3, C4.5 和 CART。
决策树通过递归地将数据集分割成更小的子集来构建树结构。具体步骤如下：

- 选择最佳特征：根据某种标准（如信息增益、基尼指数等）选择最佳特征进行分割。
- 分割数据集：根据选定的特征将数据集分成多个子集。
- 递归构建子树：对每个子集重复上述过程，直到满足停止条件（如所有样本属于同一类别，达到最大深度等）。
- 生成叶节点：当满足停止条件时，生成叶节点并赋予类别或值。

课程咨询&反馈请联系课程顾问

2. 纯度

2.1 熵 Entropy

信息熵是用来衡量“不确定性”或“混乱程度”的指标。

$$H(Y) = -\sum_{i=1}^{m} p_i \log_2 p_i$$

- p_i 是第 i 类样本所占比例。
- 熵越大，不确定性越高；熵为 0 表示数据完全纯净。

例子：二分类，正负各半，则 $H = -0.5\log_2 0.5 - 0.5\log_2 0.5 = 1$。

X	Y
Math	Yes
History	No
CS	Yes
Math	No
Math	No
CS	Yes
History	No
Math	Yes

2.2 条件熵 Conditional Entropy:

给定特征 X 后，Y 的条件熵为：

$$H(Y|X) = \sum_{\nu} p(X = \nu) \cdot H(Y|X = \nu)$$

其中 $H(Y|X = \nu)$ 是在特征 X 取值为 ν 的子集中 Y 的熵。

$$H(Y|X = \text{Math}) = -(^2/_4 * \log_2(^2/_4) + ^2/_4 * \log_2(^2/_4)) = 1$$
$$H(Y|X = \text{History}) = -(^0/_2 * \log_2(^0/_2) + * ^2/_2 \log_2(^2/_2)) = 0$$
$$H(Y|X = \text{CS}) = -(^2/_2 * \log_2(^2/_2) + ^0/_2 * \log_2(^0/_2)) = 0$$

$$\begin{aligned} H(Y|X) &= \sum_i p(X = v_i) * H(Y|X = v_i) \\ &= 0.5 * 1 + 0.25 * 0 + 0.25 * 0 \\ &= 0.5 \end{aligned}$$

2.3 信息增益 Information Gain (IG)

信息增益表示划分后信息熵 entropy 的程度。ID3 与 C4.5 选择信息增益最大的特征进行划分。

$$IG(Y|X) = H(Y) - H(Y|X)$$

$$IG(Y|X) = 1 - 0.5 = 0.5$$

2.4 基尼不纯度（Gini Impurity）

CART 算法选择基尼系数进行划分

含义：从数据集中随机抽取两个样本，其类别不一致的概率。

计算：$Gini = 1 - \sum(p_i)^2$ （p_i 是第 i 类的概率）。

策略：选择**基尼不纯度最小**（即纯度最高）的特征进行分割。

3. ID3 算法

ID3（Iterative Dichotomiser 3）采用信息增益（Information Gain）选择划分属性。

3.1 算法步骤（贪心、自顶向下递归）

分而治之（Divide-and-conquer:）思想：递归划分数据，直到子集纯净（全属同一类）

1. 从根节点开始，所有训练样本都在根节点。
2. 若当前节点样本全部属于同一类别，则停止，标记为叶节点。
3. 否则，计算每个特征的信息增益，选择信息增益最大的特征作为当前节点的划分特征。
4. 根据该特征的每个取值，将样本划分到子节点。
5. 对每个子节点递归执行上述步骤。
5. 停止条件：
   - 所有样本属于同一类。
   - 没有剩余特征可划分（用多数投票确定叶节点类别）。
   - 树深度达到预设值。

3.2 优缺点

优点

- 可解释性强：生成的规则可以直接阅读和应用。
- 实现简单：只需计算熵和信息增益。
- 能处理分类型特征，不需要归一化或哑变量编码。
- 训练速度相对较快。

缺点

- 只能处理离散特征：连续值需要先离散化（C4.5 改进为可处理连续值）。
- 对缺失值敏感：没有内置缺失值处理。
- 容易过拟合：没有剪枝，可能会生成过于复杂的树。
- 偏好取值多的特征：信息增益天生偏向选择分支多的特征。C4.5 使用信息增益率修正。
- 只能分类，不能回归。

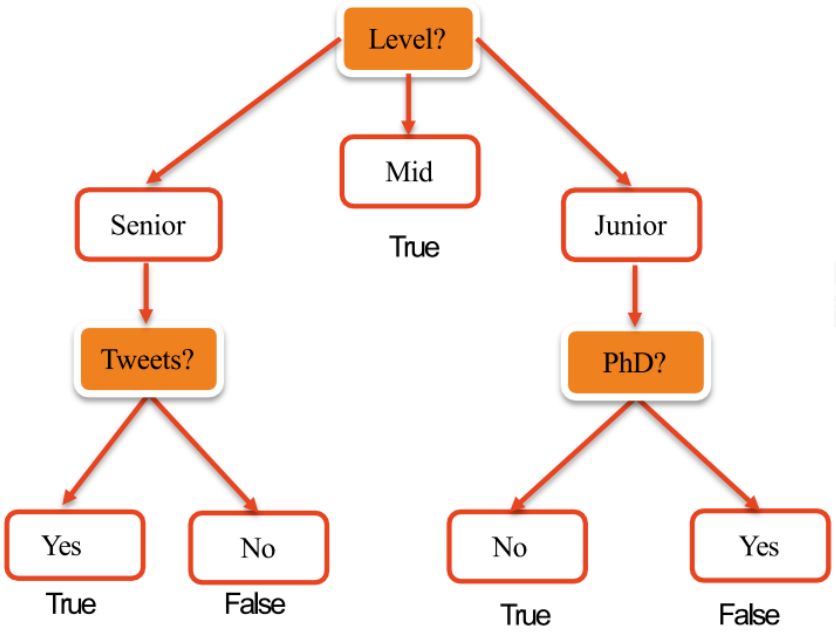
盗版必究索赔
课程咨询&反馈请联系课程顾问

3.3 示例：面试者数据集

目标：预测新申请人 A15（Senior, R, No, No）是否面试好。

Applicant	Level	Lang	Tweets	PhD	Interviewed well
A1	Senior	Java	No	No	False
A2	Senior	Java	No	Yes	False
A3	Mid	Java	No	No	True
A4	Junior	Python	No	No	True
A5	Junior	R	Yes	No	True
A6	Junior	R	Yes	Yes	False
A7	Mid	R	Yes	Yes	True
A8	Senior	Python	No	No	False
A9	Senior	R	Yes	No	True
A10	Junior	Python	Yes	No	True
A11	Senior	Python	Yes	Yes	True
A12	Mid	Python	No	Yes	True
A13	Mid	Java	Yes	No	True
A14	Junior	Python	No	Yes	False
A15	Senior	R	No	No	?

3.3.1 依靠直觉进行划分



盗版必究索赔
课程咨询&反馈请联系课程顾问

3.3.2 依靠信息增益进行划分

总样本熵

$$H(Y) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

总样本 14 个，true9 个，false5 个

$$H(\text{Interviewed well}) = H(9,5) = -({}^{9}/_{14}\log_2({}^{9}/_{14}) + {}^{5}/_{14}\log_2({}^{5}/_{14})) = 0.94$$

条件熵 Level

总样本 14 个，True 9 个，False 5 个。

$$H(\text{Interviewed well}) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} \approx 0.94$$

特征 Level：

- Senior: 5 个 (2 True, 3 False) → $H_{Senior} = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} \approx 0.97$
- Mid: 4 个 (4 True, 0 False) → $H_{Mid} = 0$
- Junior: 5 个 (3 True, 2 False) → $H_{Junior} \approx 0.97$

条件熵：

$$H(Y|\text{Level}) = \frac{5}{14} \times 0.97 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.97 \approx 0.70$$

信息增益：

$$IG = 0.94 - 0.70 = 0.24$$

类似计算其他特征：

- Tweets: IG = 0.15
- PhD: IG = 0.048
- Lang: IG = 0.029

Level 信息增益最大，故根节点按 Level 划分。

盗版必究索赔
课程咨询&反馈请联系课程顾问

3.3.3 继续划分

Level = Mid

划分后，Level=Mid 的节点已纯净（全 True），停止。

Level=Junior

	Level	Lang	Tweets	PhD	Interviewed well
4	Junior	Python	No	No	True
5	Junior	R	Yes	No	True
6	Junior	R	Yes	Yes	False
10	Junior	Python	Yes	No	True
14	Junior	Python	No	Yes	False

Level=Junior 节点（5 个样本：3 True, 2 False）需继续划分。计算 Junior 子集下各特征的 IG：

计算 Junior 子集的熵：

$$H(\text{Interviewed well}) = H(3,2) = -(^3/_5 \log_2(^3/_5) + ^2/_5 \log_2(^2/_5)) = 0.97$$

PhD 特征：

PhD=Yes: 2 个 (0 True, 2 False) → 熵 0

PhD=No: 3 个 (3 True, 0 False) → 熵 0

条件熵 = 0，IG = 0.97

Tweets 特征：

Tweets=Yes: 3 个 (2 True, 1 False) → 熵 ≈ 0.92

Tweets=No: 2 个 (1 True, 1 False) → 熵 = 1

条件熵 = 3/5×0.92 + 2/5×1 = 0.95，IG ≈ 0.02

Lang 特征：

Lang=Python: 3 个 (2 True, 1 False) → 熵 ≈ 0.92

Lang=R: 2 个 (1 True, 1 False) → 熵 = 1

条件熵 = 3/5×0.92 + 2/5×1 = 0.95，IG ≈ 0.02

因此 Junior 节点选择 PhD 划分，得到两个纯节点（PhD=Yes 全 False，PhD=No 全 True）。

Level=Senior

Level=Senior 节点（5 个：2 True, 3 False）可继续划分（可能选 Tweets 等），最终得到如 PDF 中的树。

3.3.4 预测 A15

A15 (Senior, R, No, No) 按照树：Level=Senior 分支，再根据其他特征到达叶节点，最终预测为 False。

课程咨询&反馈请联系课程顾问

4. 评估设置 EVALUATION SETUP

4.1 可靠评估的重要性

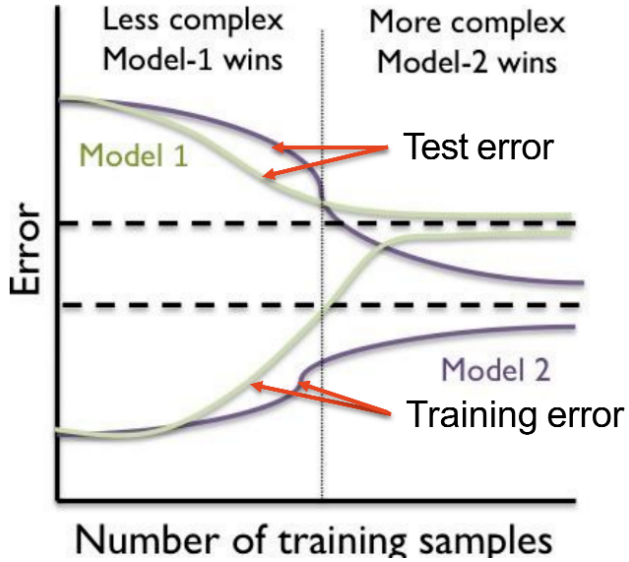
需防止过拟合，确保模型泛化能力。
常用方法：训练/测试划分、交叉验证。

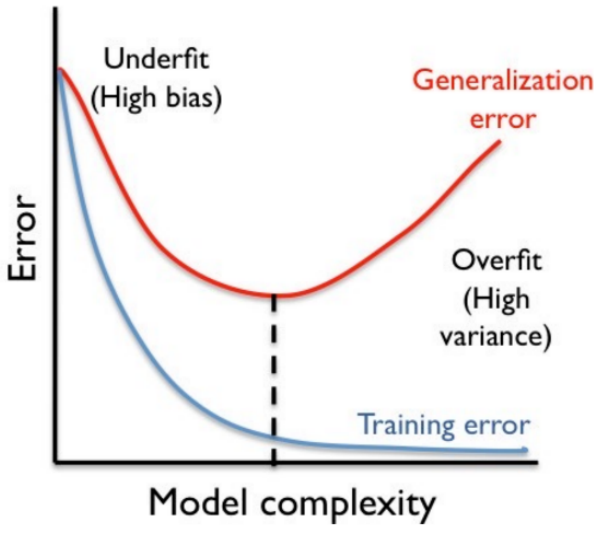
4.2 模型复杂度与数据量

- 简单模型（如线性）在数据少时可能更好；复杂模型（如深度树）需要更多数据才能发挥优势。
- 训练误差随模型复杂度降低，测试误差先降后升（过拟合）。
- 数据量小，则复杂度较低的模型 1 更优。数据量多，模型 2 最终会胜出。
- 即使拥有比现有数据更多的数据，两个模型的性能也不会有显著提升。

4.3 泛化误差

- 目标是找到测试误差最低的点（欠拟合与过拟合的平衡）。
- 必须使用独立的测试集（或交叉验证）来估计泛化误差。





4.4 剪枝 Pruning

决策树非常容易过拟合 (Overfitting)，即树长得太深、太复杂，把训练数据中的噪声也学进去了，导致在新数据上表现很差。剪枝可以降低模型复杂度。

预剪枝 (Pre-pruning)：在构建树的过程中提前停止。

- 方法：限制树的最大深度、限制叶节点最少样本数。
- 缺点：可能具有“短视”问题（贪心算法的局限）。

课程咨询&反馈请联系课程顾问

后剪枝 (Post-pruning)：先让树充分生长，再砍掉多余的分支。

- 方法：REP (错误率降低剪枝)、PEP (悲观错误剪枝)。
- 优点：通常比预剪枝效果更好，但计算成本高。

4.4 混淆矩阵 confusion matrix

混淆矩阵（Confusion Matrix）是评估分类模型性能的核心工具。它通过一个表格，清晰地展示模型预测结果与真实情况的对比，让我们能直观地看到模型"混淆"了哪些类别，从而进行更深入的诊断。

	Predicted True	Predicted False
Actual True	True Positive (TP)	False Negative (FN)
Actual False	False Positive (FP)	True Nagative (TN)

$$Accuracy=\frac{TP+TN}{TP+FN+FP+TN}$$
$$Precision=\frac{TP}{TP+FP}$$
$$Recall=\frac{TP}{TP+FN}$$
$$F1=\frac{2*Precision*Recall}{Precision+Recall}$$

准确率 (Accuracy)

公式：(TP + TN) / (TP + TN + FP + FN)

含义：模型所有预测中，正确的比例是多少？

注意：在样本类别不平衡时（如 99%正常，1%欺诈），准确率会具有欺骗性。

精确率 (Precision)

公式：TP / (TP + FP)

含义：在所有被模型预测为正例的样本中，有多少是真正的正例？它关心的是预测得"准不准"。

召回率 (Recall)

公式：TP / (TP + FN)

含义：在所有真实的正例中，模型成功找出了多少？它关心的是找得"全不全"。

F1 分数 (F1-Score)

公式：2 * (Precision * Recall) / (Precision + Recall)

含义：精确率和召回率的调和平均数。当需要同时兼顾精确率和召回率时，这是一个很好的综合指标。

只看单一的准确率指标，可能会掩盖模型的真实问题。

混淆矩阵的价值在于：

揭示类别不平衡下的真相

一个模型在欺诈检测任务中，即使总是预测"无欺诈"，也能获得 99%的准确率（如果只有 1%的欺诈样本）。但它的混淆矩阵会显示，它一个欺诈行为都没抓到（TP=0, FN=全部欺诈样本）。

指导不同场景下的决策权衡

不同类型的错误，其代价天差地别。混淆矩阵帮助我们根据业务需求做出权衡。

应用场景	哪种错误更严重?	应关注的指标	策略
癌症筛查	假负例 (FN)：漏诊会延误治疗，危及生命。	召回率 (Recall)	宁可误报（FP高），也不能漏报（FN低）。
垃圾邮件过滤	假正例 (FP)：误删重要邮件会造成重大损失。	精确率 (Precision)	宁可漏掉垃圾邮件（FN高），也不能误删正常邮件（FP低）。

课程咨询&反馈请联系课程顾问

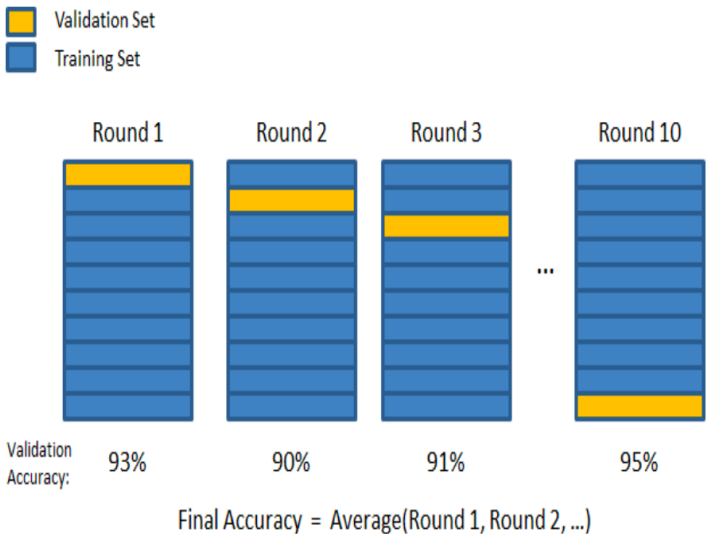
4.5 交叉验证 cross valiadation

交叉验证 核心思想是通过反复将数据集划分为训练集和验证集，来更可靠、更稳健地衡量模型在未知数据上的表现，即模型的“泛化能力”。

传统的模型评估方法是将数据一次性划分为训练集和测试集（例如 7:3），这种方法简单，但存在两大问题：
结果不稳定：评估结果高度依赖于单次随机划分。如果测试集恰好包含异常样本或类别分布不均，结果就可能产生偏差。
数据浪费：测试集只使用一次，未能充分利用所有数据来评估模型。

K 折交叉验证
操作步骤如下：

- 分割：将整个数据集随机划分为 K 个大小相似、互不重叠的子集（称为“折”，Fold）。K 值通常取 5 或 10。
- 迭代：进行 K 次训练和验证。每次选择其中 1 个折作为验证集，剩下的 K-1 个折作为训练集。
- 评估：在 K 次迭代中，每个折都会被用作一次验证集。记录下每次验证的性能指标（如准确率）。
- 平均：最后，计算 K 次验证结果的平均值，作为模型最终的 performance 评估。
- 优点：充分利用了所有数据，每个样本都参与了训练和验证；结果比单次划分更稳定。



优点：
提供更稳健的性能评估
通过多次评估取平均，交叉验证能有效减少因单次随机划分带来的偶然性波动，得到更接近模型真实性能的估计。
有效防止过拟合
过拟合是指模型在训练数据上表现很好，但在未知数据上表现很差。交叉验证通过在多个不同的数据子集上进行验证，能更好地检验模型的泛化能力，帮助发现过拟合问题。
指导模型选择与超参数调优
当需要在不同模型（如决策树 vs. 支持向量机）或不同超参数（如正则化系数）之间做选择时，交叉验证提供了一个公平的“竞技场”。我们可以比较它们在相同交叉验证设置下的平均表现，从而选出最优方案。

课程咨询&反馈请联系课程顾问

4.6 网格搜索 grid search

网格搜索（Grid Search）是机器学习中一种基础且常用的超参数优化技术。它通过系统地遍历所有预设的超参数组合，来寻找能使模型性能达到最优的那一组。

步骤：
定义参数网格：首先，你为想要调整的超参数（如学习率、树的深度）指定一系列候选值。
遍历所有组合：网格搜索会生成这些值的所有可能组合，形成一个“网格”。
训练与评估：对于网格中的每一组超参数，它都会训练一个模型，并评估其性能。
选择最优：最终，它会选出在评估中表现最好的那一组超参数。

优点：
简单直接：原理易于理解，实现方便。
结果稳定：由于是穷举，只要搜索空间设置得当，就不会错过最优解。

缺点：
计算成本高：当超参数数量增多或每个参数的候选值变多时，需要训练的模型数量会呈指数级增长，非常耗时。
不够智能：它不考虑参数之间的关联，只是机械地尝试所有组合。

交叉验证是网格搜索用来评估每一组超参数性能的“标尺”或“裁判”。

```
for name, config in models.items():
    print(f"\n🔍 训练模型: {name}")

    # 网格搜索（3折交叉验证，使用macro F1作为评分）
    grid = GridSearchCV(config['model'], config['params'],
                        cv=3, scoring='f1_macro', n_jobs=-1, verbose=0)
    grid.fit(X_train_scaled, y_train)

    best_model = grid.best_estimator_
    best_estimators[name] = best_model
    y_pred = best_model.predict(X_test_scaled)

    # 计算指标
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='macro')
    prec = precision_score(y_test, y_pred, average='macro')
    rec = recall_score(y_test, y_pred, average='macro')

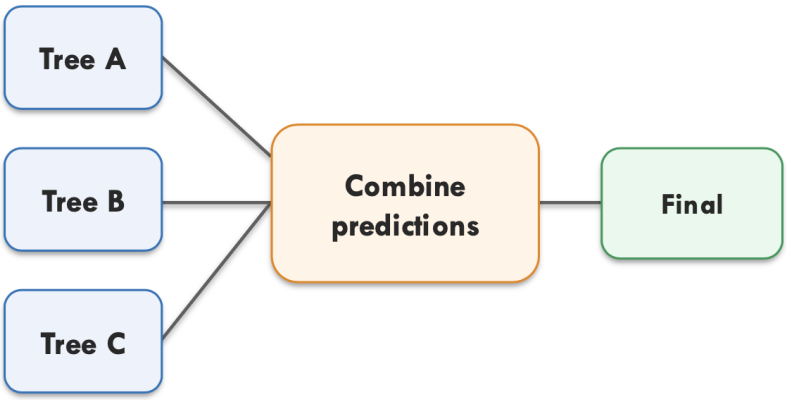
    results.append({
        'Model': name,
        'Accuracy': acc,
        'Macro F1': f1,
        'Macro Precision': prec,
        'Macro Recall': rec,
        'Best Params': grid.best_params_
    })
```

课程咨询&反馈请联系课程顾问

5. 集成树

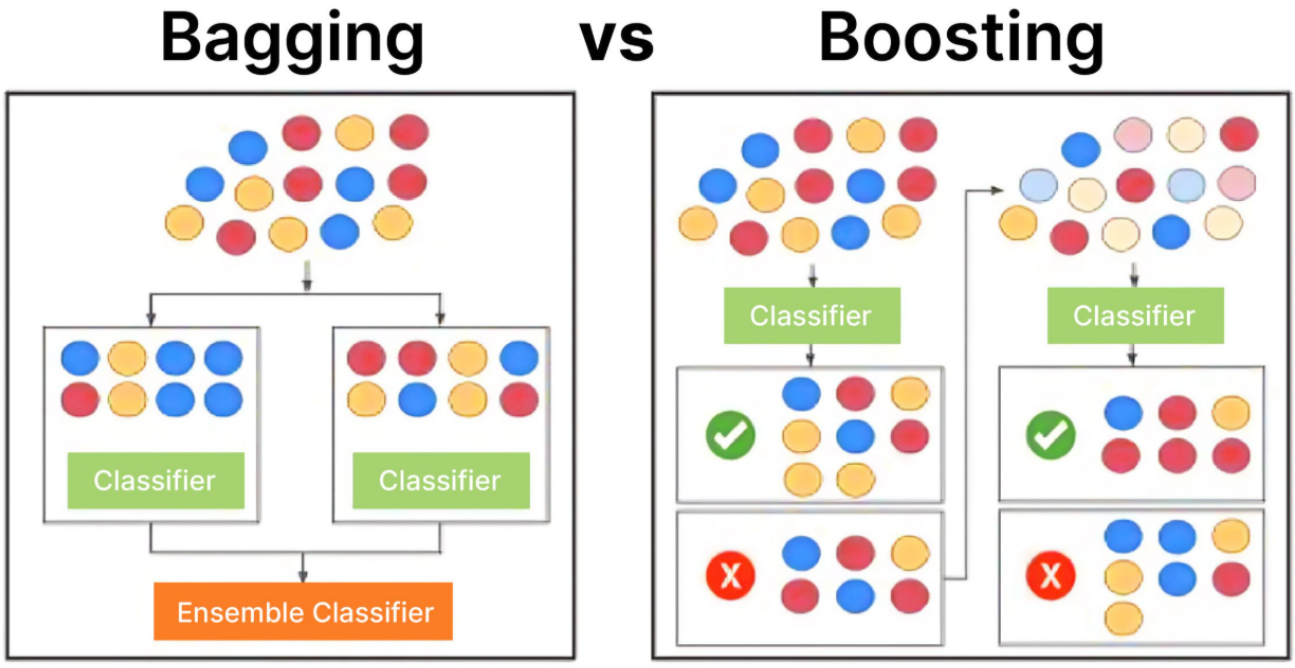
“决策树就像一位专家，非常专业，但也有自己的偏见和盲区。如果只请一位专家，万一他判断错了怎么办？”

集成决策树（Ensemble Decision Trees）通过组合多棵决策树，将多个“弱学习器”聚合成一个强大的“强学习器”，从而显著提升模型的预测能力和稳定性。根据构建这些树的方式不同，主要衍生出两大核心流派：Bagging 和 Boosting。



Bagging (Bootstrap Aggregating)
并行地训练多个基学习器，然后通过投票（分类）或平均（回归）的方式整合结果。其核心是降低方差，代表作是随机森林。

Boosting
串行地训练基学习器，每一轮都致力于修正前一轮的错误。其核心是降低偏差，代表算法包括 AdaBoost、GBDT，以及它们的优化版本 XGBoost 和 LightGBM。



课程咨询&反馈请联系课程顾问

5.1 随机森林 Random forest

随机森林的“随机”体现在两个方面，这使得森林中的每一棵树都尽可能不同（去相关性），从而让集体决策更稳健。

5.1.1 双重随机性

随机森林的精髓在于引入双重随机性，确保每棵树都不一样。

数据随机：Bootstrap 自助采样

假设你有 1000 个训练样本。对每一棵树，我们有放回地随机抽取 1000 个样本（所以有些样本会被抽到多次，有些一次都没抽到）。

- 被抽到的样本称为 袋内样本，用于训练这棵树。
- 没被抽到的样本约占 35.8%（1/e），称为 袋外样本（OOB），可以当作天然的验证集。

直觉：每棵树看到的训练数据都不一样，就像每个医生见过的病人不完全相同。

特征随机：随机子空间

普通的决策树在每个节点分裂时，会从所有特征中选最佳的那个。但在随机森林中，每棵树在分裂时只能从随机选出的部分特征中选最佳的那个。

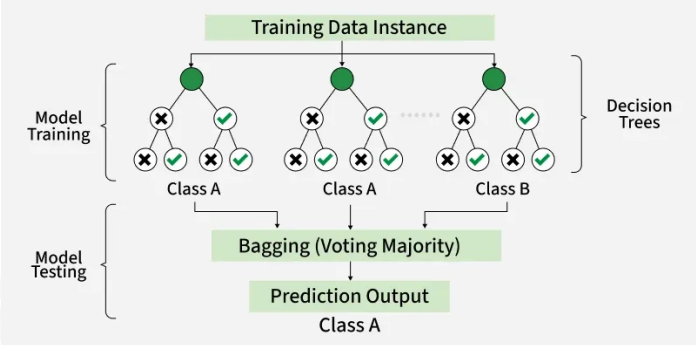
- 对分类问题，通常随机选 $\sqrt{p}$ 个特征（p 是特征总数）。
- 对回归问题，通常随机选 p/3 个特征。

直觉：如果每棵树都能用所有特征，那它们可能都选了最强的那个特征做根节点，所有树长得很像，投票没有意义。强制只让看部分特征，各棵树的“视角”不同，才有可能学到不同的规律。

5.1.2 算法流程

构建随机森林的过程可以概括为以下四步：

1. 采样：从原始训练集中进行 N 次 Bootstrap 采样，生成 N 个不同的训练子集。
2. 建树：对每个子集，构建一棵决策树。
   - 在生长过程中，每到一个节点，随机选择 m 个特征，从中选出最优分裂点。
   - 注意：随机森林的树通常不进行剪枝，让它们充分生长，以保持低偏差。
3. 重复：重复上述步骤，生成由 T 棵树组成的“森林”。
4. 集成：
   - 分类问题：采用多数投票 (Majority Voting)，得票最多的类别为最终结果。
   - 回归问题：采用平均法 (Averaging)，取所有树预测值的平均数。



课程咨询&反馈请联系课程顾问

5.1.3 为什么随机森林不容易过拟合？

单一决策树如果很深，几乎可以把训练集完全拟合（准确率 100%），但在测试集上表现差。但随机森林通过两个手段控制方差：

- Bagging：取平均/投票平滑了各棵树的极端输出。
- 特征随机：打破了最强的特征对每棵树的“垄断”，各棵树犯的错不相关，平均后就抵消了。

一句话总结：每棵树单独看可能都在过拟合，但它们的错误方向不同，彼此抵消后，整体模型的泛化能力反而很强。

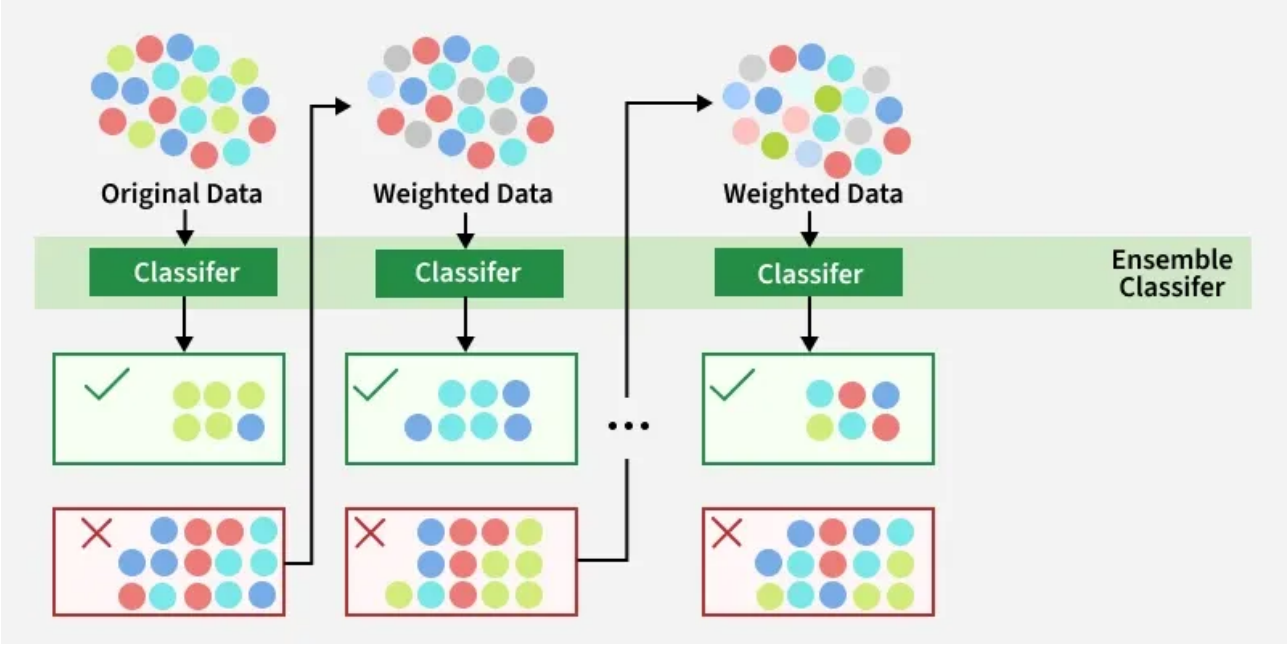
“100 个偏激的专家，如果偏激的方向各不相同，投票结果反而可能是最公允的。”

5.2 AdaBoost (Adaptive Boosting)

AdaBoost 是 Boosting 家族的开创者，其思想非常直观。

核心原理：调整样本权重

1. 初始化时，所有训练样本的权重相同。
2. 训练第一个弱分类器（通常是决策树桩，即只有一层的决策树）。
3. 根据分类结果，提高被分错样本的权重，降低被分对样本的权重。
4. 训练下一个弱分类器，此时它会更加关注那些权重高的“难啃的骨头”。
5. 重复此过程，最终将所有弱分类器根据其表现（错误率）进行加权投票，得到最终结果。



主要优点：

分类精度高，尤其在二分类问题上表现出色。

算法简单，易于实现。

局限性：

对噪声和异常值非常敏感，因为算法会持续给错分的样本增加权重，可能导致模型过度[illegible]这些噪声点。

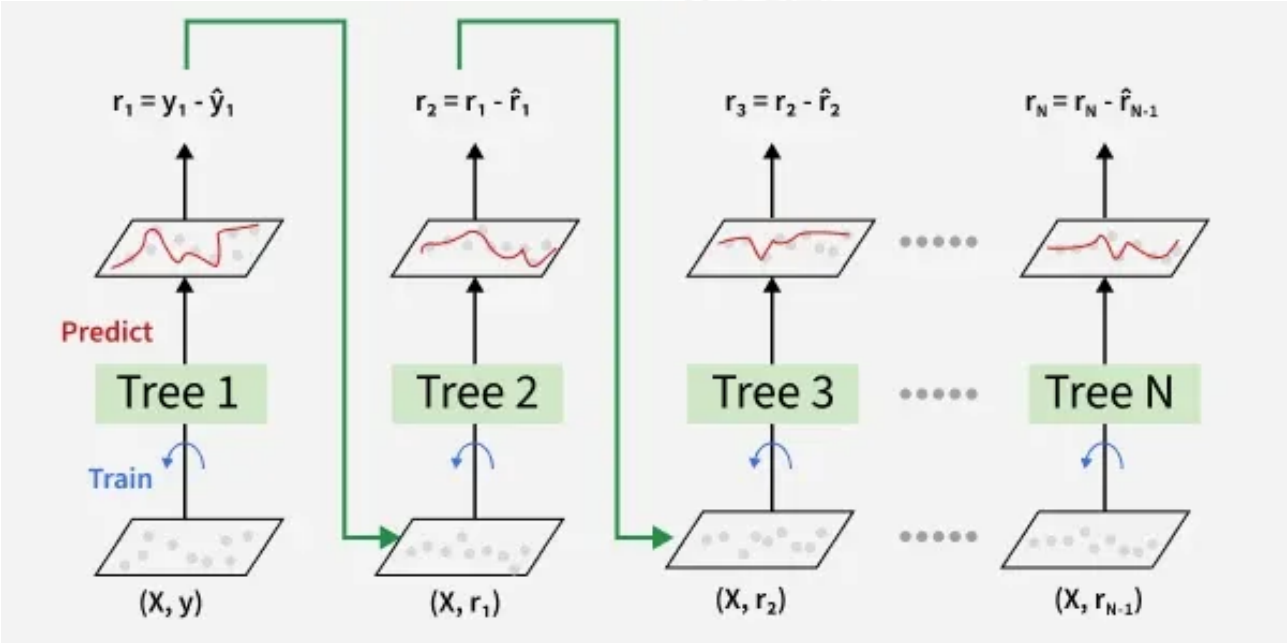
5.3 GBDT (Gradient Boosting Decision Tree)

GBDT 将 Boosting 的思想与梯度下降优化算法相结合，使其理论基础更加坚实，应用也更加广泛。
核心原理：拟合残差（负梯度）

与 AdaBoost 调整样本权重不同，GBDT 的每一棵新树都是在拟合前一轮模型的残差（即真实值与当前模型预测值之间的差距）。从优化的角度看，这个过程等价于在损失函数空间中进行梯度下降，新树拟合的是损失函数的负梯度。

主要优点：
拟合能力强，精度高：通过逐步逼近最优解，在很多任务上表现优异。
灵活性高：可以使用多种不同的损失函数（如回归中的均方误差，分类中的对数似然损失）。

局限性：
串行训练，难以并行化，训练速度较慢。
对超参数（如树的数量、学习率、树的深度）比较敏感，需要仔细调参。



5.4 LightGBM (Light Gradient Boosting Machine)

LightGBM 是微软在 GBDT 框架基础上推出的高效实现，专为处理海量数据和高维特征而设计，可以看作是 GBDT 的“工程优化升级版”。

核心原理：
Leaf-wise 生长策略：不同于传统 GBDT/XGBoost 的 Level-wise（按层生长，像盖楼），LightGBM 每次选择分裂增益最大的叶子节点进行分裂（像挖井），在相同分裂次数下能获得更低的误差，但可能生成更深的树。

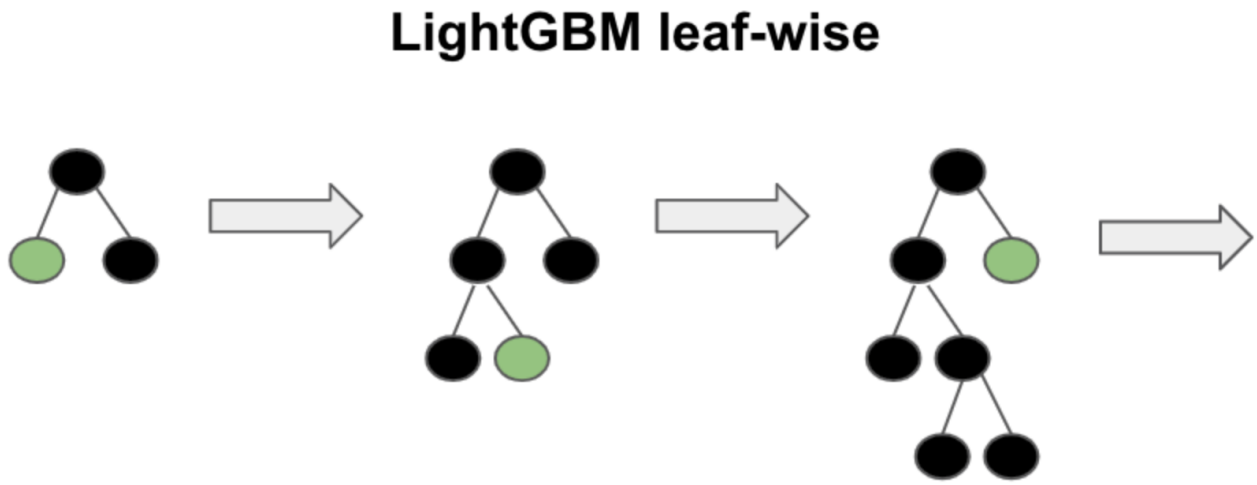
单边梯度采样 (GOSS)：保留梯度绝对值大的样本（信息量大），对梯度小的样本进行随机采样，这能在几乎不损失精度的前提下，大幅减少参与计算的数据量。

课程咨询&反馈请联系课程顾问

互斥特征捆绑（EFB）：将许多互斥的（不会同时取非零值的）稀疏特征捆绑成一个特征，从而有效降低特征维度。

主要优点：
训练速度极快：比 XGBoost 快很多，尤其适合大规模数据。
内存占用低：通过直方图算法等技术，显著降低了内存消耗。
支持类别特征：无需对类别特征进行独热编码（One-Hot Encoding），避免了维度爆炸。

局限性：
在小数据集上可能不如 XGBoost 稳定。
Leaf-wise 策略若不加限制（如 max_depth），容易导致过拟合。



算法	所属流派	核心原理	主要优点	主要缺点
随机森林	Bagging	样本与特征双重随机，并行训练，投票/平均	抗过拟合，鲁棒性强，易于使用	模型复杂，解释性差
AdaBoost	Boosting	迭代调整样本权重，关注错分样本	分类精度高，算法简单	对噪声敏感
GBDT	Boosting	串行训练，每棵树拟合前一轮的残差（负梯度）	拟合能力强，精度高	训练慢，难并行，需调参
LightGBM	Boosting (优化)	基于 GBDT，采用 Leaf-wise、GOSS、EFB 等技术加速	速度极快，内存占用低	小数据集表现可能不佳，易过拟合

盗版必究索赔
课程咨询&反馈请联系课程顾问

6. 课后习题

案例一：信息增益与决策树分裂（手算）

数据集： 判断是否适合户外运动（Outdoor），基于两个特征：天气（Weather）和风力（Windy）。

1. 计算整个数据集关于目标变量 `Outdoor` 的熵 $H(Y)$。
2. 分别计算以 `Weather` 和 `Windy` 为分裂属性的条件熵 $H(Y \mid X)$。
3. 计算两个特征各自的信息增益 $IG(Y \mid X)$。
4. 根据信息增益，应该选择哪个特征作为根节点？为什么？
5. 如果选择 `Weather` 作为根节点，请画出根节点分裂后的子节点，并判断每个子节点是否还需要继续分裂（假设当节点中所有样本的 `Outdoor` 一致时停止）。

Day	Weather	Windy	Outdoor
1	Sunny	False	No
2	Sunny	False	No
3	Overcast	False	Yes
4	Rain	False	Yes
5	Rain	True	No
6	Rain	True	No
7	Overcast	True	Yes
8	Sunny	True	Yes
9	Sunny	False	Yes
10	Rain	False	Yes
11	Sunny	True	No
12	Overcast	True	Yes

1. 总样本数 12， `Outdoor` = Yes 有 6 个，No 有 6 个。
   $H(Y) = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1$
2. `Weather` 取值：Sunny (4个样本: Yes=1, No=3), Overcast (3个: Yes=3), Rain (5个: Yes=2, No=3)
   $H(Y|Sunny) = -0.25 \log_2 0.25 - 0.75 \log_2 0.75 \approx 0.811$
   $H(Y|Overcast) = 0$, $H(Y|Rain) = -0.4 \log_2 0.4 - 0.6 \log_2 0.6 \approx 0.971$
   条件熵 = $\frac{4}{12} \times 0.811 + \frac{3}{12} \times 0 + \frac{5}{12} \times 0.971 \approx 0.675$
   $IG(Weather) = 1 - 0.675 = 0.325$
   类似计算 `Windy`: True (5个: Yes=2, No=3) → 熵0.971; False (7个: Yes=4, No=3) → 熵0.985;
   条件熵 = $\frac{5}{12} \times 0.971 + \frac{7}{12} \times 0.985 \approx 0.979$
   $IG(Windy) \approx 0.021$
3. Weather 的信息增益更大，选它作根节点。
4. 根节点 Weather 分三支：
   - Sunny: 4个样本 (1Y,3N) → 不纯，需继续分裂。
   - Overcast: 全 Y → 叶节点 (Outdoor = Yes)。
   - Rain: 5个样本 (2Y,3N) → 不纯，需继续分裂。

盗版必究索赔

案例二：混淆矩阵与评估指标计算

场景： 一个二分类模型用于预测病人是否患有某种疾病（Disease = Positive / Negative）。模型在测试集上的预测结果如下表（混淆矩阵）：

1. 计算模型的准确率 (Accuracy)。
2. 计算精确率 (Precision) 与召回率 (Recall)。
3. 计算 F1 分数。
4. 如果该疾病患病率很低（大约 10%），且漏诊（实际有病却预测为无病）的代价极高，你会更看重精确率还是召回率？为什么？
5. 假设我们调整分类阈值，使得模型变得更容易预测为 Positive（即更多样本被判为有病）。那么精确率和召回率会如何变化？（只需定性说明）

	Predicted Positive	Predicted Negative
Actual Positive	90	30
Actual Negative	20	380

1. Accuracy = (90+380)/(90+30+20+380) = 470/520 ≈ 0.904
2. Precision = 90/(90+20) = 90/110 ≈ 0.818；Recall = 90/(90+30) = 90/120 = 0.75
3. F1 = 2 * (0.818*0.75)/(0.818+0.75) ≈ 0.782
4. 重视召回率，因为漏诊的代价比误诊更高，希望尽可能找出所有真正的病人。
5. 阈值降低 → 更多样本被判为 Positive → 精确率通常会下降（因为引入了更多假阳性），召回率会上升（更多真正的阳性被捉住）。

盗版必究索赔

案例三：交叉验证与模型选择（计算题）

背景： 你训练了两种不同的决策树模型（Tree A 和 Tree B），并采用 5 折交叉验证评估其性能。以下是它们在每一折上的准确率：

1. 计算 Tree A 和 Tree B 的平均准确率和标准差。
2. 哪个模型的平均性能更好？哪个模型更稳定（即波动更小）？
3. 如果你需要部署一个对性能波动敏感的模型，你会选择哪个？为什么？
4. 假设你使用留出法（单次训练/测试划分，比例 80/20）得到的结果是 Tree A 更好，但交叉验证显示 Tree B 的平均性能更优。你认为哪个评估结果更可信？解释原因。

Fold	Tree A Accuracy	Tree B Accuracy
1	0.82	0.80
2	0.79	0.83
3	0.85	0.81
4	0.81	0.84
5	0.83	0.82

1. Tree A 平均 = (0.82+0.79+0.85+0.81+0.83)/5 = 0.82；标准差 ≈ 0.021
   Tree B 平均 = (0.80+0.83+0.81+0.84+0.82)/5 = 0.82；标准差 ≈ 0.014
2. 平均性能相同；Tree B 更稳定（标准差更小）。
3. 选 Tree B，因为其性能在不同数据子集上更一致，泛化能力更可靠。
4. 交叉验证的结果更可信，因为它使用了多次不同划分，减少了因单次划分偶然性导致的偏差。

盗版必究索赔
课程咨询&反馈请联系课程顾问